

UT4 Estructuras Definidas por el Usuario en JavaScript

Desarrollo Web en entorno Cliente (DEW)

Profesora: Yaiza Guanche Pérez
yguaper@gobiernodecanarias.org



Tabla de Contenidos

DEW

1. Funciones predefinidas del lenguaje
2. Funciones predefinidas por el usuario
3. Llamadas a funciones
4. Definición de funciones
5. Arrays
6. Inicialización de Arrays
7. Recorrido de Arrays
8. Creación de Objetos
9. Definición de métodos y propiedades

Funciones predefinidas del lenguaje

DEW

eval (cadena [objeto])

Cadena: es una cadena que representa una expresión, sentencia o secuencia de sentencias en JavaScript La expresión puede incluir variables y propiedades de objetos existentes.

Objeto

un argumento opcional si se especifica, la evaluación restringe al contexto del objeto especificado.

```
eval (new String(" 2 2 "));; // devuelve un objeto String q contiene 2 2
```

```
eval(" 2 2 "));; // devuelve 4
```


Funciones predefinidas del lenguaje

DEW

parseInt(cadena, base)

Cadena: una cadena que representa el valor que se desea convertir.

Base: un entero que representa la base de la mencionada cadena.

Si la cadena comienza por 0 x" es hexadecimal base 16 si comienza por 0 octal base 8 si comienza por cualquier otro valor, la base es 10 decimal

Si se encuentra con un carácter que no corresponde con la base especificada, devuelve NaN

Ejemplo:

parseInt 0 x 7 10 No es base 10 devuelve NaN

Funciones predefinidas del lenguaje

DEW

parseFloat (cadena)

Cadena: una cadena que representa el valor que se desea convertir.
Si el primer carácter no se puede convertir a número,
devuelve NaN.

isNaN(valor) // is Not a Number

Valor: el valor que se desea evaluar.

Intenta convertir el argumento a número, si no puede, devuelve
true, en caso contrario, false.

Funciones predefinidas del lenguaje

DEW

isFinite(numero)

Valor: valor que se desea evaluar.

Si el argumento es NaN, infinito positivo o negativo devuelve false, en caso contrario true.

```
isFinite(0); // verdadero
```

```
isFinite(5-2); // verdadero
```

```
isFinite("123"); // falso
```

```
isFinite("Hola"); // falso
```


Funciones predefinidas del usuario

DEW

Son un conjunto de sentencias que realizan una tarea especificada.
No se ejecuta cuando es declarada sino cuando es invocada.

Consta de:

La palabra reservada **function** para definirla.

El **nombre** que queremos darle a nuestra función.

Los **argumentos** que podemos usar internamente (opcional)

La función puede devolver o no un valor, que estaría indicado a continuación de la palabra reservada **return**.

Definiciones. Llamadas a Funciones

DEW

SINTAXIS DE UNA FUNCIÓN:

```
fuction nombre_funcion (param1, param2, param3...) {  
}  
function saludar() {  
    alert("Hola");  
}  
saludar();
```

¿Qué ocurre aquí en nuestro navegador?

Definiciones. Llamadas a Funciones

DEW

SINTAXIS DE UNA FUNCIÓN:

```
fuction nombre_funcion (param1, param2, param3...) {  
}  
function saludar() {  
    alert("Hola");  
}  
let saludo = saludar();  
alert(saludo);
```

¿Qué ocurre aquí en nuestro navegador?

Definiciones. Llamadas a Funciones

DEW

SINTAXIS DE UNA FUNCIÓN:

```
fuction nombre_funcion (param1, param2, param3...) {  
}  
function saludar() {  
    alert("Hola");  
}  
let saludo = saludar;  
alert(saludo);
```

¿Qué ocurre aquí en nuestro navegador?

Definiciones. Llamadas a Funciones

DEW

SINTAXIS DE UNA FUNCIÓN:

```
function nombre_funcion (param1, param2, param3...) {  
}
```

```
function saludar (nombre) {  
    alert("Hola " + nombre);  
}
```

```
saludar("Yaiza");    //llamamos a nuestra función con parámetros
```


Definiciones. Llamadas a Funciones

DEW

SINTAXIS DE UNA FUNCIÓN:

```
fuction nombre_funcion (param1, param2, param3...) {  
}
```

```
function saludar (nombre) {  
    return "Hola " + nombre;  
}
```

```
let saludo = saludar("Yaiza");           //llamamos a nuestra función  
alert(saludo);
```

Definiciones. Llamadas a Funciones

DEW

Función anónima: aquellas funciones que no han sido declaradas con un nombre. Pueden ejecutar una función o darle un parámetro a otra función.

```
function () {  
}
```

```
let producto = function (a,b) { return a * b };
```

```
let resultado = producto (5, 6); // invocamos la función  
alert (resultado);    // la variable
```

Características:

- Almacena en una misma variable múltiples valores: valores primitivos, objetos, etc...
- Se referencian con un índice numérico que comienza en 0.
- Un objeto se referencia con un nombre. (Queremos ver un elemento cuyo nombre es ...)
- Puede almacenar diferentes tipos de datos.

Inicialización de Arrays

- Crear un array: `var nombreArray = [valor1, valor2, valor3, ...];`

`let animales = [];`

- ¿Cómo los asignamos?

`let animales = ["perro", "gato", "loro"];`

- Otras formas:

`let animales = [];`

`animales[0]="perro";`

`animales[1]="gato";`

`animales[2]="loro";`

`let animales = [];`

`animales.push("perro");`

`animales.push("gato");`

`animales.push("loro");`

Recorrido de Arrays

DEW

- **Mostrar:**

```
alert(animales); //muestra todo el array en ventana separado por comas.
```

```
document.write (animales); //muestra todo el array en el documento  
                        // separado por comas.
```

```
// Muestra cada elemento del array de forma individual.
```

```
for (i = 0; i < animales.length; i++) {  
    document.write ("Animales[" + i + "]" + animales[i] + "<br/>");  
}
```

Creación de Objetos

DEW

1. Definir un objeto simple utilizando un literal

```
let Edificio = {  
  calle = "García Prieto";  
  numero = 58;  
  codigoPostal = 15706;  
}
```


Creación de Objetos

DEW

2. Definir y crear un objeto simple utilizando la palabra new.

```
let Edificio = new Object();
```

```
Edificio.calle = "García Prieto";
```

```
Edificio.numero = 58;
```

```
Edificio.CodigoPostal = 15706;
```

Creación de Objetos

DEW

3. Definir un constructor de un objeto y crear objetos del tipo construido.

```
function Edificio (c, num, cp) {  
  this.calle = c;  
  this.numero = num;  
  this.codigoPostal = cp;  
}
```

```
Class Edificio {  
  constructor (c, num, cp) {  
    this.calle = c;  
    this.numero = num;  
    this.codigoPostal = cp;  
  }  
}
```

Dos formas de hacer el constructor, aunque como hemos visto en JavaScript no hace falta definir una clase, pero se puede hacer.

El mismo método para instanciarlo y crearlo.

```
let edificioA = new Edificio ("García Prieto", 58, 15706);
```

Definición de métodos y propiedades

DEW

Propiedades: se definen en modo nombre:valor, pueden ser datos primitivos, otros objetos o ser funciones. (vistas anteriormente en los constructores)

Métodos: acciones que se pueden ejecutar sobre un objeto.

```
modificarCalle (c) {    // ejemplo de un método que modifica el valor de una propiedad
    this.calle = c;      // del objeto
};

imprimeCodigoPostal () { // ejemplo de un método que retorna una propiedad del
    return this.codigoPostal;
};
```

Nota: En JavaScript también existen clases anónimas como vimos en clase con las funciones.